

Indian Institute of Technology Kharagpur

AUTUMN Semester, 2026–2027

COMPUTER SCIENCE AND ENGINEERING

Computer Organization and Architecture Laboratory

Verilog Assignment 1

August 2026

***Instructions:** This is a group assignment. Each group must implement the complete Verilog design, write a suitable testbench, and demonstrate the working design and its behavioral simulation to a Teaching Assistant for evaluation. The assignment must be submitted as a group submission through Moodle.*

Question 1: Hierarchical Design of a 4-bit Combinational Logic Unit

Design and simulate a **4-bit combinational logic unit** in Verilog. The complete design must follow a hierarchical module structure in which different functionalities are implemented as separate Verilog modules and connected together in a top-level module.

1. Half Adder Module

Create a module named

`half_adder`

with the following interface:

Inputs	Outputs
A, B	Sum, Carry

The module must implement a 1-bit Half Adder.

2. Full Adder Module

Create a module named

`full_adder`

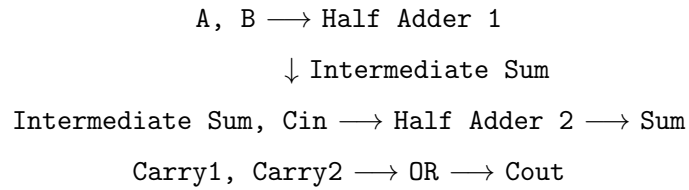
with the following interface:

Inputs	Outputs
A, B, Cin	Sum, Cout

The Full Adder must be constructed using **two instances of the half_adder module**.
The connections are as follows:

1. The first Half Adder receives A and B.
2. The Sum output of the first Half Adder is connected to one input of the second Half Adder.
3. Cin is connected to the other input of the second Half Adder.
4. The Sum output of the second Half Adder becomes the Sum output of the Full Adder.
5. The two Carry outputs are combined to generate Cout.

The structure is therefore:



3. 4-bit Ripple-Carry Adder Module

Create a module named

ripple_carry_adder_4bit

having:

Inputs	Output
A[3:0], B[3:0]	Sum[4:0]

The module must be constructed using **four instances of the full_adder module**.

The carry output of each Full Adder must be connected to the carry input of the next Full Adder:

$$FA0 \xrightarrow{C_1} FA1 \xrightarrow{C_2} FA2 \xrightarrow{C_3} FA3$$

The connections are:

Module	A Input	B Input	Carry Input
FA0	A[0]	B[0]	0
FA1	A[1]	B[1]	Carry from FA0
FA2	A[2]	B[2]	Carry from FA1
FA3	A[3]	B[3]	Carry from FA2

The individual Sum outputs produce Sum[3:0], while the final carry output from FA3 becomes Sum[4].

4. Bitwise Logic Modules

Create three separate modules for the bitwise operations:

`and_4bit`, `or_4bit`, `xor_4bit`.

Each module receives two 4-bit inputs:

`A[3:0]`, `B[3:0]`

and generates a 5-bit output:

`Y[4:0]`.

The required operations are:

Module	Operation
<code>and_4bit</code>	Bitwise A AND B
<code>or_4bit</code>	Bitwise A OR B
<code>xor_4bit</code>	Bitwise A XOR B

Since these operations produce a 4-bit result, `Y[4]` must be zero.

5. Multiplexer Module

Create a module named

`mux4to1_5bit`

having four 5-bit data inputs:

`I0[4:0]`, `I1[4:0]`, `I2[4:0]`, `I3[4:0]`,

a 2-bit select input

`sel[1:0]`,

and a 5-bit output

`Y[4:0]`.

The multiplexer must select one of the four inputs according to:

sel	Selected Input
00	I0
01	I1
10	I2
11	I3

6. Top-Level Logic Unit

Create the top-level module

`logic_unit`

with the following interface:

Signal	Width	Direction
A	4 bits	Input
B	4 bits	Input
sel	2 bits	Input
Y	5 bits	Output

The module must perform the following operation depending on `sel`:

sel	Operation
00	Addition: $A + B$
01	Bitwise AND
10	Bitwise OR
11	Bitwise XOR

The top-level module must **not implement these operations directly**. Instead, it must instantiate and connect:

- one `ripple_carry_adder_4bit`;
- one `and_4bit`;
- one `or_4bit`;
- one `xor_4bit`; and
- one `mux4to1_5bit`.

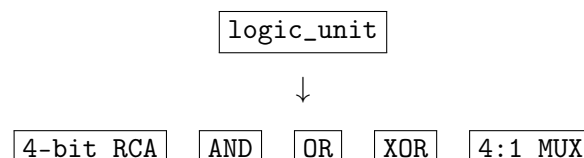
The outputs of the four operation modules must be connected to the four inputs of the multiplexer as follows:

MUX Input	Connected Module
I0	<code>ripple_carry_adder_4bit</code>
I1	<code>and_4bit</code>
I2	<code>or_4bit</code>
I3	<code>xor_4bit</code>

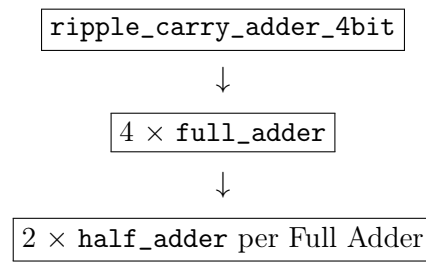
The `sel` input of the top-level module must be connected to the select input of the multiplexer. The output of the multiplexer becomes the final output `Y`.

7. Complete Design Hierarchy

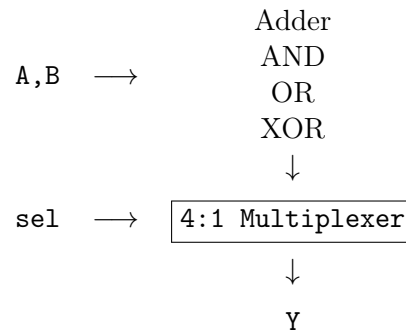
The top-level organization is:



The adder hierarchy is:



Thus, the functional data flow of the top-level module is:



8. Verilog Design Requirement

The design must follow the **hierarchical module structure** specified above.

Each functionality must be implemented in its corresponding Verilog module, and larger modules must instantiate the required lower-level modules.

Students may use suitable combinational Verilog constructs inside the individual modules, including:

- gate primitives;
- continuous assignment using **assign**;
- **always @(*)** blocks;
- **if-else** statements; and
- **case** statements.

However, the complete functionality must not be written as a single monolithic Verilog module.

The design must be purely combinational. No clocked or sequential logic is required.

9. Testbench and Demonstration

Write a testbench for the top-level **logic_unit** module.

The testbench must:

1. test all four values of **sel**;
2. use multiple combinations of **A** and **B**;
3. verify the output for addition, AND, OR, and XOR operations; and
4. generate the behavioral simulation waveform in Vivado.

The waveform must show at least:

A, B, sel, Y.

Demonstration is mandatory. Each group must demonstrate the working Verilog design and behavioral simulation to a Teaching Assistant. Students may also be asked to explain the module hierarchy, the carry propagation in the Ripple-Carry Adder, and the selection of the final output through the multiplexer.

10. Submission

Submit all Verilog design source files and the testbench in a single zipped folder named

`VerilogAssignment1_{Group_Number}.zip`.

The submission must contain the modules:

- `half_adder`;
- `full_adder`;
- `ripple_carry_adder_4bit`;
- `and_4bit`;
- `or_4bit`;
- `xor_4bit`;
- `mux4to1_5bit`;
- `logic_unit`; and
- the testbench for `logic_unit`.

Each source file must contain the group number, names and roll numbers of all group members, assignment number, and date of submission.